Set-A

1. **Given a string s containing only the characters**
   (,), {, }, [,]
   determine if the input string is valid.
   A string is valid if:
   (a) Open brackets are closed by the same type of brackets.
   (b) Open brackets are closed in the correct order.
   (c) Every closing bracket has a corresponding opening bracket of the same type.
   Examples:
   (a) Input: s = ”()”
   Output: true
   (b) Input: s = ”()[]{}”
   Output: true

One of the solution:

```
bool isValid(char* s) {
    int n = strlen(s);
    char* stack = (char*)malloc(n * sizeof(char));
    int top = -1;

    for (int i = 0; i < n; i++) {
        char c = s[i];
        if (c == '(' || c == '{' || c == '[') {
            stack[++top] = c; // push opening bracket
        } else {
            if (top == -1) return false; // no opening bracket

            char topChar = stack[top--]; // pop from stack
            if ((c == ')' && topChar != '(') ||
                (c == '}' && topChar != '{') ||
                (c == ']' && topChar != '[')) {
                free(stack);
                return false;
            }
        }
    }

    bool valid = (top == -1); // stack must be empty
    free(stack);
```

```
    return valid;
}
```

****

2. **You are given an array of strings tokens that represents an arithmetic expression in Reverse Polish Notation (RPN) or Postfix expression.**
   Evaluate the expression and return an integer that represents the value of the expression.
   Example 1: Input: tokens = [“2”, “1”, “+”, “3”, “*”] Output: 9 Explanation: ((2 + 1) × 3) = 9

One of the solution:

```
int evalRPN(char **tokens, int tokensSize) {
    int *stack = (int *)malloc(tokensSize * sizeof(int));
    int top = -1;

    for (int i = 0; i < tokensSize; i++) {
        char *t = tokens[i];
        if (isNumber(t)) {
            stack[++top] = atoi(t); // push number
        } else {
            int b = stack[top--]; // pop second operand
            int a = stack[top--]; // pop first operand
            if (strcmp(t, "+") == 0) stack[++top] = a + b;
            else if (strcmp(t, "-") == 0) stack[++top] = a - b;
            else if (strcmp(t, "*") == 0) stack[++top] = a * b;
            else if (strcmp(t, "/") == 0) stack[++top] = a / b; // truncates toward 0
        }
    }

    int result = stack[top];
```

```
    free(stack);
    return result;
}
```

****

3. **Design a stack that, in addition to push(x), pop(), and peek(), also supports retrieving the minimum element in constant time.**
   Implement the following operations:
   (a) push(x) → Insert element x into the stack.
   (b) pop() → Remove the element from the top of the stack and return it. If the stack is empty, return -1.
   (c) peek() → Return the top element without removing it. If the stack is empty, return -1.
   (d) getMin() → Return the minimum element in the stack. If the stack is empty, return -1.
   Examples:
   (a) Input: push(5), push(3), push(7), getMin(), pop(), getMin()
   Output: 3, 3
   Explanation:
   Stack after pushes → [7, 3, 5]
   Minimum element = 3
   After pop() (removes 7) → stack = [3, 5], minimum = 3

One of the solution:

```
// Check empty
int isEmpty(Stack* stack) {
    return stack->top == -1;
}

// Push
void pushStack(Stack* stack, int x) {
    if (stack->top == stack->capacity - 1) return; // overflow
    stack->arr[++stack->top] = x;
}
```

```
// Pop
int popStack(Stack* stack) {
    if (isEmpty(stack)) return -1;
    return stack->arr[stack->top--];
}

// Peek
int peekStack(Stack* stack) {
    if (isEmpty(stack)) return -1;
    return stack->arr[stack->top];
}

// -------- MinStack Implementation --------
typedef struct {
    Stack* mainStack;
    Stack* minStack;
} MinStack;

// Create MinStack
MinStack* createMinStack(int capacity) {
    MinStack* ms = (MinStack*)malloc(sizeof(MinStack));
    ms->mainStack = createStack(capacity);
    ms->minStack = createStack(capacity);
    return ms;
}

// Push
void push(MinStack* ms, int x) {
    pushStack(ms->mainStack, x);
    if (isEmpty(ms->minStack) || x <= peekStack(ms->minStack)) {
        pushStack(ms->minStack, x);
```

```
    } else {
        pushStack(ms->minStack, peekStack(ms->minStack));
    }
}

// Pop
int pop(MinStack* ms) {
    if (isEmpty(ms->mainStack)) return -1;
    popStack(ms->minStack);
    return popStack(ms->mainStack);
}

// Peek
int peek(MinStack* ms) {
    return peekStack(ms->mainStack);
}

// Get Min
int getMin(MinStack* ms) {
    return peekStack(ms->minStack);
}
```

****

Set-B

1. **Given a 0-indexed string word and a character ch, reverse the segment of word that starts at index 0 and ends at the index of the first occurrence of ch (inclusive). If the character ch does not exist in word, do nothing. Return the resulting string.**
   Example 1:
   Input: word = "abcdefd", ch = "d" Output: "dcbaefd" Explanation: The first occurrence of "d"
   is at index 3. Reverse the part of word from 0 to 3 (inclusive), the resulting string is "dcbaefd".

One of the solution:

```
// Function to reverse a part of string
void reverseSegment(char* str, int start, int end) {
    while (start < end) {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

// Main function
char* reversePrefix(char* word, char ch) {
    int n = strlen(word);
    int idx = -1;

    // Find first occurrence of ch
    for (int i = 0; i < n; i++) {
        if (word[i] == ch) {
            idx = i;
            break;
        }
    }

    // If character not found, return word as is
    if (idx == -1) return word;

    // Reverse from 0 to idx
    reverseSegment(word, 0, idx);
    return word;
```

```
}
```

****

2. **You are given an array of strings tokens that represents an arithmetic expression in Reverse Polish Notation (RPN) or Postfix expression.**
   Evaluate the expression and return an integer that represents the value of the expression.
   Example 1: Input: tokens = [“2”, “1”, “+”, “3”, “*”] Output: 9 Explanation: ((2 + 1) × 3) = 9

One of the solution:

```
int evalRPN(char **tokens, int tokensSize) {
    int *stack = (int *)malloc(tokensSize * sizeof(int));
    int top = -1;

    for (int i = 0; i < tokensSize; i++) {
        char *t = tokens[i];
        if (isNumber(t)) {
            stack[++top] = atoi(t); // push number
        } else {
            int b = stack[top--]; // pop second operand
            int a = stack[top--]; // pop first operand
            if (strcmp(t, "+") == 0) stack[++top] = a + b;
            else if (strcmp(t, "-") == 0) stack[++top] = a - b;
            else if (strcmp(t, "*") == 0) stack[++top] = a * b;
            else if (strcmp(t, "/") == 0) stack[++top] = a / b; // truncates toward 0
        }
    }

    int result = stack[top];
    free(stack);
```

```
    return result;
}
```

****

3. **The next greater element of some element x in an array is the first greater element that is to the**
   right of x in the same array.
   You are given two distinct 0-indexed integer arrays nums1 and nums2, where nums1 is a subset of
   nums2.
   For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and determine the
   next greater element of nums2[j] in nums2. If there is no next greater element, then the answer for
   this query is -1.
   Return an array ans of length nums1.length such that ans[i] is the next greater element as described
   above.
   Example 1:
   Input: nums1 = [4,1,2], nums2 = [1,3,4,2]
   Output: [-1,3,-1]

One of the solution:

```
int* nextGreaterElement(int* nums1, int nums1Size, int* nums2, int nums2Size, int* returnSize) {
    *returnSize = nums1Size;
    int *ans = (int*)malloc(nums1Size * sizeof(int));

    for (int i = 0; i < nums1Size; i++) {
        int found = 0;
        int nextGreater = -1;

        // Find nums1[i] in nums2
        for (int j = 0; j < nums2Size; j++) {
            if (nums2[j] == nums1[i]) {
                found = 1;
                // Look for next greater
                for (int k = j + 1; k < nums2Size; k++) {
                    if (nums2[k] > nums2[j]) {
                        nextGreater = nums2[k];
```

```
                    break;
                }
            }
            break;
        }
    }
    ans[i] = nextGreater;
}

return ans;
}
```

****

Set-C

1. **Given a string s containing only the characters**
   (,), {, }, [,]
   determine if the input string is valid.
   A string is valid if:
   (a) Open brackets are closed by the same type of brackets.
   (b) Open brackets are closed in the correct order.
   (c) Every closing bracket has a corresponding opening bracket of the same type.
   Examples:
   (a) Input: s = ”()”
   Output: true
   (b) Input: s = ”()[]{}”
   Output: true

One of the solution:

```
bool isValid(char* s) {
    int n = strlen(s);
    char* stack = (char*)malloc(n * sizeof(char));
    int top = -1;

    for (int i = 0; i < n; i++) {
        char c = s[i];
        if (c == '(' || c == '{' || c == '[') {
            stack[++top] = c; // push opening bracket
        } else {
            if (top == -1) return false; // no opening bracket

            char topChar = stack[top--]; // pop from stack
            if ((c == ')' && topChar != '(') ||
                (c == '}' && topChar != '{') ||
                (c == ']' && topChar != '[')) {
                free(stack);
                return false;
            }
        }
    }

    bool valid = (top == -1); // stack must be empty
    free(stack);
```

```
    return valid;
}
```

****

2. **Implement a stack using a singly linked list**

   Examples:

   (a) Input: push(10), push(20), push(30), peek(), pop(), peek(), isEmpty()

   Output: 30, 30, 20, false

One of the solution:

```
// Create a new stack
Stack* createStack() {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    stack->top = NULL;
    return stack;
}

// Push element onto stack
void push(Stack* stack, int x) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    newNode->next = stack->top;
    stack->top = newNode;
}

// Pop element from stack
int pop(Stack* stack) {
    if (stack->top == NULL) {
        return -1; // empty stack
```

```
    }
    Node* temp = stack->top;
    int val = temp->data;
    stack->top = stack->top->next;
    free(temp);
    return val;
}

// Peek top element
int peek(Stack* stack) {
    if (stack->top == NULL) {
        return -1;
    }
    return stack->top->data;
}

// Check if stack is empty
bool isEmpty(Stack* stack) {
    return stack->top == NULL;
}
```

****

3. **The next greater element of some element x in an array is the first greater element that is to the right of x in the same array. You are given two distinct 0-indexed integer arrays nums1 and nums2, where nums1 is a subset of nums2.**
   For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and determine the next greater element of nums2[j] in nums2. If there is no next greater element, then the answer for this query is -1.
   Return an array ans of length nums1.length such that ans[i] is the next greater element as described
   above.

Example 1:

Input: nums1 = [4,1,2], nums2 = [1,3,4,2]

One of the solution:

```
int* nextGreaterElement(int* nums1, int nums1Size, int* nums2, int nums2Size, int* returnSize) {
    *returnSize = nums1Size;
    int *ans = (int*)malloc(nums1Size * sizeof(int));

    for (int i = 0; i < nums1Size; i++) {
        int found = 0;
        int nextGreater = -1;

        // Find nums1[i] in nums2
        for (int j = 0; j < nums2Size; j++) {
            if (nums2[j] == nums1[i]) {
                found = 1;
                // Look for next greater
                for (int k = j + 1; k < nums2Size; k++) {
                    if (nums2[k] > nums2[j]) {
                        nextGreater = nums2[k];
                        break;
                    }
                }
                break;
            }
        }
        ans[i] = nextGreater;
    }

    return ans;
}
```

****

Set-D

1. **Given a 0-indexed string word and a character ch, reverse the segment of word that starts at index 0 and ends at the index of the first occurrence of ch (inclusive). If the character ch does not exist in word, do nothing. Return the resulting string.**
   Example 1:
   Input: word = "abcdefd", ch = "d" Output: "dcbaefd" Explanation: The first occurrence of "d"
   is at index 3. Reverse the part of word from 0 to 3 (inclusive), the resulting string is "dcbaefd".

One of the solution:

```
// Function to reverse a part of string
void reverseSegment(char* str, int start, int end) {
    while (start < end) {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

// Main function
char* reversePrefix(char* word, char ch) {
    int n = strlen(word);
    int idx = -1;

    // Find first occurrence of ch
    for (int i = 0; i < n; i++) {
        if (word[i] == ch) {
            idx = i;
            break;
        }
    }

    // If character not found, return word as is
    if (idx == -1) return word;

    // Reverse from 0 to idx
    reverseSegment(word, 0, idx);
    return word;
```

```
}
```

****

2. **Implement a stack using a singly linked list**

   Examples:

   (a) Input: push(10), push(20), push(30), peek(), pop(), peek(), isEmpty()

   Output: 30, 30, 20, false

One of the solution:

```
// Create a new stack
Stack* createStack() {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    stack->top = NULL;
    return stack;
}

// Push element onto stack
void push(Stack* stack, int x) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    newNode->next = stack->top;
    stack->top = newNode;
}

// Pop element from stack
int pop(Stack* stack) {
    if (stack->top == NULL) {
        return -1; // empty stack
    }
```

```
    Node* temp = stack->top;
    int val = temp->data;
    stack->top = stack->top->next;
    free(temp);
    return val;
}

// Peek top element
int peek(Stack* stack) {
    if (stack->top == NULL) {
        return -1;
    }
    return stack->top->data;
}

// Check if stack is empty
bool isEmpty(Stack* stack) {
    return stack->top == NULL;
}
```

****

3. **Design a stack that, in addition to push(x), pop(), and peek(), also supports retrieving the minimum element in constant time.**
   Implement the following operations:
   (a) push(x) → Insert element x into the stack.
   (b) pop() → Remove the element from the top of the stack and return it. If the stack is empty, return -1.
   (c) peek() → Return the top element without removing it. If the stack is empty, return -1.
   (d) getMin() → Return the minimum element in the stack. If the stack is empty, return -1.
   Examples:
   (a) Input: push(5), push(3), push(7), getMin(), pop(), getMin()
   Output: 3, 3

Explanation:
Stack after pushes → [7, 3, 5]
Minimum element = 3
After pop() (removes 7) → stack = [3, 5], minimum = 3

One of the solution:

```
// Check empty
int isEmpty(Stack* stack) {
    return stack->top == -1;
}

// Push
void pushStack(Stack* stack, int x) {
    if (stack->top == stack->capacity - 1) return; // overflow
    stack->arr[++stack->top] = x;
}

// Pop
int popStack(Stack* stack) {
    if (isEmpty(stack)) return -1;
    return stack->arr[stack->top--];
}

// Peek
int peekStack(Stack* stack) {
    if (isEmpty(stack)) return -1;
    return stack->arr[stack->top];
}

// -------- MinStack Implementation --------
typedef struct {
```

```
    Stack* mainStack;
    Stack* minStack;
} MinStack;

// Create MinStack
MinStack* createMinStack(int capacity) {
    MinStack* ms = (MinStack*)malloc(sizeof(MinStack));
    ms->mainStack = createStack(capacity);
    ms->minStack = createStack(capacity);
    return ms;
}

// Push
void push(MinStack* ms, int x) {
    pushStack(ms->mainStack, x);
    if (isEmpty(ms->minStack) || x <= peekStack(ms->minStack)) {
        pushStack(ms->minStack, x);
    } else {
        pushStack(ms->minStack, peekStack(ms->minStack));
    }
}

// Pop
int pop(MinStack* ms) {
    if (isEmpty(ms->mainStack)) return -1;
    popStack(ms->minStack);
    return popStack(ms->mainStack);
}

// Peek
int peek(MinStack* ms) {
```

```
    return peekStack(ms->mainStack);
}

// Get Min
int getMin(MinStack* ms) {
    return peekStack(ms->minStack);
}
```

****